
kalman_filter

Release 0.1

Chheng Lydiya

Feb 28, 2026

CONTENTS

1	Introductory	3
1.1	Introduction	3
1.2	Probability	4
2	Kalman Filter	7
2.1	Kalman Filter Implementation	7
2.2	Extended Kalman Filter	10

Add your content using reStructuredText syntax. See the [reStructuredText](#) documentation for details.

INTRODUTORY

1.1 Introduction

These notes focus on **Kalman filtering** and **optimal state estimation**, covering both the underlying theory and practical methods for estimating the state of dynamic systems from noisy measurements. Kalman filters are widely used in engineering, robotics, navigation, finance, and many other fields where systems change over time and measurements are uncertain.

1.1.1 Scope of These Notes

The goal of this document is to provide a structured set of notes that:

- Explains the mathematics and intuition behind **state estimation**.
- Covers **linear system models**, Gaussian noise assumptions, and their role in deriving optimal estimates.
- Derives the **Kalman filter equations** from first principles.
- Explores **extensions and variations**, including the Extended Kalman Filter (EKF), Unscented Kalman Filter (UKF), and Kalman smoothing techniques.
- Provides examples and derivations to build intuition about uncertainty, prediction, and measurement updating.

These notes are intended for readers who already have a basic understanding of probability and random variables. As such, rather than starting from scalar random variables and elementary probability concepts, the discussion begins at **multiple random variables**. This approach is more relevant for practical systems, where different state components—such as position, velocity, or sensor biases—interact and must be estimated simultaneously.

1.1.2 Why Start at Multiple Random Variables

Real-world systems are rarely one-dimensional. A robot's motion, an aircraft's navigation, or a financial portfolio all involve **interdependent quantities** that cannot be treated independently. Modeling these requires **vector random variables**, which allow us to describe not only the expected value of each component but also the **covariance between components**, capturing how uncertainties are correlated.

By starting here, these notes focus on the mathematical tools and concepts that directly feed into **state-space modeling** and the **Kalman filter**, including:

- **Mean vectors** and **covariance matrices** to represent uncertainty in multiple dimensions.
- **Joint Gaussian distributions** and their properties, which are foundational for deriving Kalman updates.
- **Linear transformations** of random vectors, which appear in system dynamics and measurement models.
- **Conditional distributions**, which are central to updating state estimates when new measurements become available.

1.1.3 Structure and Approach

The notes are organized to gradually build from these foundational concepts toward a full understanding of the Kalman filter:

1. **Multiple Random Variables** – vector notation, covariance, joint Gaussian distributions, and linear transformations.
2. **State-Space Models** – linear system representation, process and measurement models, and noise characterization.
3. **Kalman Filter** – derivation of prediction and update equations, intuition, and properties.
4. **Extensions** – nonlinear systems (EKF, UKF), smoothing, and other variations.
5. **Applications and Examples** – practical scenarios, implementation tips, and illustrative computations.

By framing the notes in this way, readers can focus on the **core ideas of state estimation** without being slowed by material they already know, while still maintaining a complete and coherent understanding of Kalman filtering.

1.1.4 Why These Notes Matter

Kalman filtering is not just a theoretical exercise—it is a **powerful tool** for making sense of uncertain, dynamic systems. Understanding how to model uncertainty, combine predictions with measurements, and optimize estimates is a skill that applies across engineering, data science, and decision-making contexts. These notes are designed to provide both the **theoretical foundation** and **practical insight** to develop that skill.

1.2 Probability

Attention

We won't be covering all probability theory. This section focuses on multiple random variables and stochastic processes.

1.2.1 Multiple Random Variables

CDF

If X and Y are random variables (RVs), their cumulative distribution functions (CDFs) are

$$F_X(x) = P(X \leq x)$$

$$F_Y(y) = P(Y \leq y)$$

The joint CDF is

$$F_{X,Y}(x, y) = P(X \leq x, Y \leq y)$$

Properties of the joint CDF

$$0 \leq F_{X,Y}(x, y) \leq 1$$

$$\lim_{x \rightarrow -\infty} F_{X,Y}(x, y) = 0, \quad \lim_{y \rightarrow -\infty} F_{X,Y}(x, y) = 0$$

$$\lim_{x, y \rightarrow \infty} F_{X,Y}(x, y) = 1$$

$$\text{if } a \leq b \text{ and } c \leq d, \quad F_{X,Y}(a, c) \leq F_{X,Y}(b, d)$$

$$P(a < X \leq b, c < Y \leq d) = F_{X,Y}(b, d) - F_{X,Y}(a, d) - F_{X,Y}(b, c) + F_{X,Y}(a, c)$$

Joint PDF

When X and Y are (jointly) continuous, the joint probability density function (PDF) is the mixed partial derivative of the joint CDF:

$$f_{X,Y}(x,y) = \frac{\partial^2}{\partial x \partial y} F_{X,Y}(x,y)$$

Equivalently,

$$F_{X,Y}(x,y) = \int_{-\infty}^x \int_{-\infty}^y f_{X,Y}(u,v) dv du$$

Properties of the joint PDF

$$f_{X,Y}(x,y) \geq 0$$

$$\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f_{X,Y}(x,y) dx dy = 1$$

$$P(a < X \leq b, c < Y \leq d) = \int_a^b \int_c^d f_{X,Y}(x,y) dy dx$$

Marginal PDFs

$$f_X(x) = \int_{-\infty}^{\infty} f_{X,Y}(x,y) dy$$

$$f_Y(y) = \int_{-\infty}^{\infty} f_{X,Y}(x,y) dx$$

For discrete random variables, replace PDFs with PMFs and integrals with sums.

KALMAN FILTER

2.1 Kalman Filter Implementation

With continuous state-space model state-space equation:

$$\begin{aligned}\dot{x} &= Ax + Bu + \sqrt{Q_C}v(t) \\ y &= Cx + \sqrt{R}w(t)\end{aligned}$$

Discretization the model into:

$$\begin{aligned}x_k &= x_{k-1} + T_s(Ax_{k-1} + Bu_{k-1}) + \sqrt{Q_c T_s} v_{k-1} \\ y_k &= Cx_k\end{aligned}$$

```
c = [1, 0]

function x_dot = condyn(x, u, A, B)
    x_dot = A * x + B * u
end

function x_k = disdyn(x, u, A, B, T_s, Q_c)
    n = length(x);
    x_k = x + T_s * condyn(x, u, A, B) + \sqrt{Q_c * T_s} * random(n, 1)
end

function y = measure(x, c)
    y = c * x
end
```

Note

condyn = continuous dynamic

disdyn = discrete dynamic

A = transition matrix of continuous state

B = input matrix of continuous state

```
for k = 2:N_sample
    % true state anf measurement
    u = sin(t(k - 1));
    [xk, ~] = disdyn(x_est_store(:, k - 1), u, A, B, T_s);
    x_store(:, k) = xk + \sqrt{Q_c * T_s} * randn(2, 1);
    y_store(1, k) = measure(x_store(:, k), c) + \sqrt{R} * randn(1, 1);
end
```

(continues on next page)

(continued from previous page)

```

% kalman filter
% next state prediction and error covariance update
[xk, Ak] = disdyn(x_est_store(:, k - 1), u, A, B, T_s);
x_est_store(:, k) = xk;
p = Ak * p * Ak' + Q_d;

% estimate measurement, cross covariance, auto covariance, kalman gain
y_est = measure(x_est_store(:, k), c);
p_xy = p * c;
p_yy = c * p * c' + r; % innovation covariance
k = p_xy/p_yy; % kalman gain
x_est_store(:, k) = x_est_store(:, k) + k * y_store(1, k) - y_est; % update_
↪estimate
p = (eye(2) - k * c) * p; update error covariance
end

```

2.1.1 Python implementation

```

import numpy as np
import matplotlib.pyplot as plt

# -----
# Parameters
# -----
Ts = 0.1          # sampling time
N_sample = 100    # number of samples

A = np.array([[0, 1],
               [0, 0]]) # continuous-time A
B = np.array([[0],
               [1]])   # continuous-time B
c = np.array([[1, 0]]) # measurement matrix

Q_c = np.array([[0.01, 0],
                 [0, 0.01]]) # continuous process noise
R = 0.1 # measurement noise

Q_d = Q_c * Ts # discrete process noise

# -----
# Storage initialization
# -----
x_store = np.zeros((2, N_sample))
y_store = np.zeros(N_sample)
x_est_store = np.zeros((2, N_sample))
p = np.eye(2) # initial error covariance

x_store[:, 0] = [0, 0]
x_est_store[:, 0] = [0, 0]

t = np.arange(N_sample) * Ts

# -----
# Functions
# -----

```

(continues on next page)

(continued from previous page)

```

def condyn(x, u, A, B):
    """Continuous dynamics"""
    return A @ x + B.flatten() * u

def disdyn(x, u, A, B, Ts):
    """Discrete dynamics and linearized Ak"""
    xk = x + Ts * condyn(x, u, A, B)
    Ak = np.eye(len(x)) + Ts * A # linearized discrete-time transition
    return xk, Ak

def measure(x, c):
    """Measurement function"""
    return c @ x

# -----
# Simulation loop
# -----
for k in range(1, N_sample):
    u = np.sin(t[k-1])

    # True system + measurement noise
    xk, _ = disdyn(x_est_store[:, k-1], u, A, B, Ts)
    x_store[:, k] = xk + np.random.multivariate_normal([0,0], Q_c*Ts)
    y_store[k] = measure(x_store[:, k], c) + np.random.normal(0, np.sqrt(R))

    # Kalman filter prediction
    xk_pred, Ak = disdyn(x_est_store[:, k-1], u, A, B, Ts)
    x_est_store[:, k] = xk_pred
    p = Ak @ p @ Ak.T + Q_d

    # Measurement update
    y_pred = measure(x_est_store[:, k], c)
    p_xy = p @ c.T
    p_yy = c @ p @ c.T + R
    K = p_xy / p_yy # Kalman gain
    x_est_store[:, k] = x_est_store[:, k] + K.flatten() * (y_store[k] - y_pred)
    p = (np.eye(2) - K @ c) @ p

# -----
# Plot results
# -----
plt.figure(figsize=(10,6))
plt.plot(t, x_store[0, :], label='True position')
plt.plot(t, x_est_store[0, :], label='Estimated position', linestyle='--')
plt.plot(t, y_store, label='Measurements', linestyle=':', alpha=0.7)
plt.xlabel('Time [s]')
plt.ylabel('Position')
plt.title('Kalman Filter Tracking (Updated Implementation)')
plt.legend()
plt.grid(True)
plt.show()

```

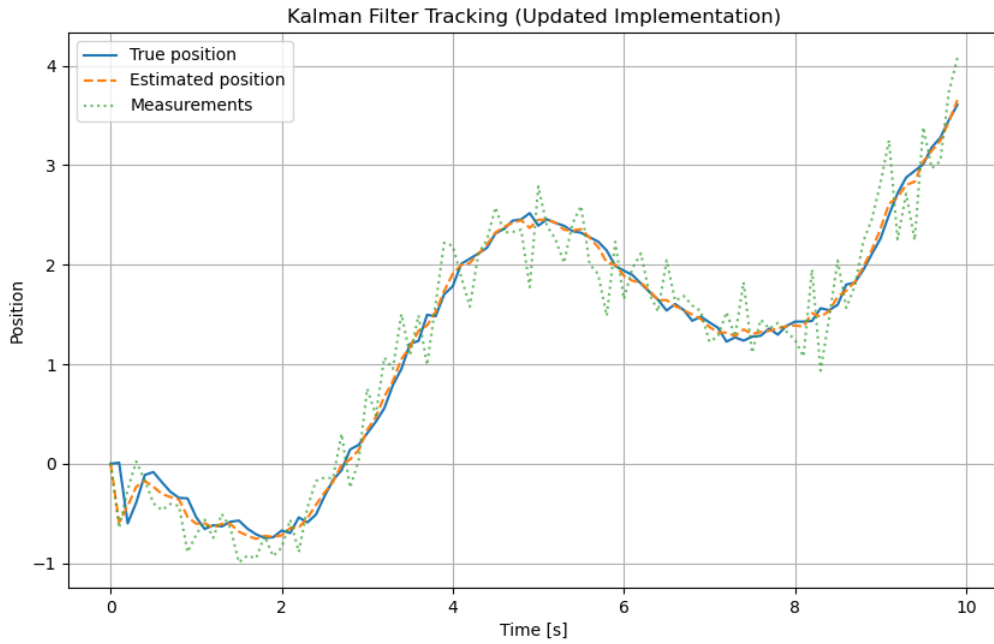


Fig. 1: Kalman filter simulation results: true state, measurements, and estimated state.

2.2 Extended Kalman Filter

2.2.1 Review (Kalman filter) on covariance prediction and correction

For a nonlinear dynamic system:

$$x_k = f(x_{k-1}, u_{k-1}) \quad (\text{nonlinear state transition})$$

$$y_k = g(x_k, u_k) \quad (\text{nonlinear measurement})$$

- Time update (prediction):

$$P_{k|k-1} = A_k P_{k-1|k-1} A_k^T + Q_d$$

Note: A constant linear matrix A does not exist for nonlinear systems; instead we linearize about the current estimate and use a time-varying Jacobian A_k .

- Measurement (correction):

$$P_{k|k} = (I - K_k C_k) P_{k|k-1}$$

The key step in the Extended Kalman Filter (EKF) is computing the Jacobians A_k and C_k (the linearizations of f and g about the current estimate):

$$A_k = \left. \frac{\partial f}{\partial x} \right|_{x=\hat{x}_{k-1|k-1}}, \quad C_k = \left. \frac{\partial g}{\partial x} \right|_{x=\hat{x}_{k|k-1}}$$

The Jacobian A_k can be written elementwise as

$$A_k = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}_{x=\hat{x}_{k-1|k-1}}$$

and similarly for C_k using partial derivatives of g .

and similarly for C_k using partial derivatives of g (measurements $y \in \mathbb{R}^m$):

$$C_k = \begin{bmatrix} \frac{\partial g_1}{\partial x_1} & \cdots & \frac{\partial g_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial g_m}{\partial x_1} & \cdots & \frac{\partial g_m}{\partial x_n} \end{bmatrix}_{x=\hat{x}_{k|k-1}}$$

For differential equations (continuous $\rightarrow$ discrete):

$$\dot{x} = f(x, u) \Rightarrow x_k \approx x_{k-1} + T_s f(x_{k-1}, u_{k-1})$$

The discrete-time Jacobian (linearized about the estimate) is:

$$A_{k-1} = I + T_s \left. \frac{\partial f}{\partial x} \right|_{x=\hat{x}_{k-1|k-1}}$$

The estimated state vector (column form) is written as:

$$\hat{x}_{k|k} = \begin{bmatrix} \hat{x}_1 \\ \hat{x}_2 \\ \vdots \\ \hat{x}_n \end{bmatrix}$$

Example

$$\dot{\omega} = -a \omega + 1.5 \sin(t) + b + \sqrt{0.1} v(t)$$

Let

$$x_1 = \omega, \quad x_2 = a, \quad x_3 = b$$

Then the continuous-time dynamics for the state components are

$$\begin{aligned} \dot{x}_1 &= -x_2 x_1 + 1.5 \sin(t) + x_3 + \sqrt{0.1} v_1(t) \\ \dot{x}_2 &= 0 + \sqrt{0.01} v_2(t) \\ \dot{x}_3 &= 0 + \sqrt{0.01} v_3(t) \end{aligned}$$

or in compact form

$$\dot{x} = f(x, t) + \sqrt{Q_c} v(t)$$

with

$$f(x, t) = \begin{bmatrix} -x_2 x_1 + 1.5 \sin(t) + x_3 \\ 0 \\ 0 \end{bmatrix}$$

and the process-noise term

$$\sqrt{Q_c} v(t) = \begin{bmatrix} \sqrt{0.1} & 0 & 0 \\ 0 & \sqrt{0.01} & 0 \\ 0 & 0 & \sqrt{0.01} \end{bmatrix} \begin{bmatrix} v_1(t) \\ v_2(t) \\ v_3(t) \end{bmatrix}$$

The continuous-time Jacobian:

$$A_c = \partial f / \partial x = \begin{bmatrix} -x_2 & -x_1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

The discretized linearization (Euler, sample time T_s) is

$$A_{k-1} = I_3 + T_s A_c \big|_{x=\hat{x}_{k-1|k-1}}$$

If we measure only the angular rate ω , the measurement model is

$$y = \omega = x_1 = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} x$$

```

% ekf_example.m
% Extended Kalman Filter example (3-state) using Euler discretization
function ekf_example()
    Ts = 0.1;
    N = 200;
    t = 0:Ts:Ts*(N-1);

    % true initial state
    x_true = [0.5; 0.1; 0.2];
    % initial estimate
    x = [0;0;0];
    P = diag([0.5,0.5,0.5]);

    Qc = diag([0.1, 0.01, 0.01]);
    R = 0.05;

    rng('default');

    for k = 1:N
        tk = t(k);
        % simulate true system (Euler)
        w = mvnrnd([0 0 0], Qc)';
        x_true = x_true + Ts * f_cont(x_true, tk) + sqrt(Ts)*w;

        % measurement
        v = sqrt(R)*randn();
        y = x_true(1) + v;

        % predict
        [x_pred, P_pred] = ekf_predict_matlab(x, P, Ts, Qc, tk);

        % update
        [x, P] = ekf_update_matlab(x_pred, P_pred, y, R);
    end

    disp('Final true state:'); disp(x_true');
    disp('Final estimate:'); disp(x');
end

% Plot results
figure;
tt = 0:Ts:Ts*(N-1);
subplot(3,1,1);
plot(tt, true_states(1,:), 'b-', tt, est_states(1,:), 'r--');
legend('true omega', 'est omega'); ylabel('omega');
subplot(3,1,2);
plot(tt, true_states(2,:), 'b-', tt, est_states(2,:), 'r--');
legend('true a', 'est a'); ylabel('a');
subplot(3,1,3);
plot(tt, true_states(3,:), 'b-', tt, est_states(3,:), 'r--');
legend('true b', 'est b'); ylabel('b'); xlabel('time (s)');

function dx = f_cont(x, t)
    omega = x(1); a = x(2); b = x(3);
    dx = [-a*omega + 1.5*sin(t) + b; 0; 0];
end

```

(continues on next page)

(continued from previous page)

```

function A = A_cont(x, t)
    omega = x(1); a = x(2);
    A = [-a, -omega, 1; 0,0,0; 0,0,0];
end

function [x_pred, P_pred] = ekf_predict_matlab(x, P, Ts, Qc, t)
    x_pred = x + Ts * f_cont(x, t);
    A = eye(3) + Ts * A_cont(x, t);
    P_pred = A * P * A' + Ts * Qc;
end

function [x_upd, P_upd] = ekf_update_matlab(x_pred, P_pred, y, R)
    C = [1 0 0];
    S = C * P_pred * C' + R;
    K = (P_pred * C') / S;
    x_upd = x_pred + K * (y - C * x_pred);
    P_upd = (eye(3) - K * C) * P_pred;
end

```

```

import numpy as np

def f(x, t):
    # continuous dynamics converted to discrete via Euler (sample time applied_
    ↪ outside)
    # x = [omega, a, b]
    omega, a, b = x
    dx1 = -a * omega + 1.5 * np.sin(t) + b
    dx2 = 0.0
    dx3 = 0.0
    return np.array([dx1, dx2, dx3])

def g(x):
    # measurement: y = omega
    return np.array([x[0]])

def A_continuous(x, t):
    # Jacobian of f wrt x (continuous-time)
    omega, a, b = x
    return np.array([[ -a, -omega, 1.0],
                     [0.0, 0.0, 0.0],
                     [0.0, 0.0, 0.0]])

def C_jacobian(x):
    # Jacobian of g wrt x
    return np.array([[1.0, 0.0, 0.0]])

def ekf_predict(x, P, Ts, Qc, t):
    # discrete-time predict using Euler discretization
    # x_{k} = x_{k-1} + Ts * f(x_{k-1}, t)
    x_pred = x + Ts * f(x, t)
    A = np.eye(3) + Ts * A_continuous(x, t)
    P_pred = A @ P @ A.T + Ts * Qc
    return x_pred, P_pred

def ekf_update(x_pred, P_pred, y, R):

```

(continues on next page)

(continued from previous page)

```

    C = C_jacobian(x_pred)
    S = C @ P_pred @ C.T + R
    K = P_pred @ C.T @ np.linalg.inv(S)
    x_upd = x_pred + (K @ (y - g(x_pred))).reshape(-1)
    P_upd = (np.eye(len(x_pred)) - K @ C) @ P_pred
    return x_upd, P_upd

def simulate_and_run(Ts=0.1, N=200):
    t = 0.0
    # true initial state
    x_true = np.array([0.5, 0.1, 0.2])
    # initial estimate
    x = np.array([0.0, 0.0, 0.0])
    P = np.diag([0.5, 0.5, 0.5])

    # continuous process noise covariance (Qc) and measurement noise (R)
    Qc = np.diag([0.1, 0.01, 0.01])
    R = np.array([[0.05]])

    history = []

    for k in range(N):
        # simulate true system (Euler discrete approx)
        w = np.random.multivariate_normal(np.zeros(3), Qc) * np.sqrt(Ts)
        x_true = x_true + Ts * f(x_true, t) + w

        # measurement
        v = np.random.normal(0, np.sqrt(R[0,0]))
        y = np.array([x_true[0] + v])

        # EKF predict
        x_pred, P_pred = ekf_predict(x, P, Ts, Qc, t)

        # EKF update
        x, P = ekf_update(x_pred, P_pred, y, R)

        history.append((t, x_true.copy(), x.copy(), P.copy(), y.copy()))
        t += Ts

    return history

if __name__ == '__main__':
    hist = simulate_and_run()
    # print final results
    t, x_true, x_est, P, y = hist[-1]
    print('Final time: {:.2f}s'.format(t))
    print('True state: ', x_true)
    print('Estimated: ', x_est)
    print('Measurement: ', y)

    # Prepare and save plots
    try:
        import matplotlib
        matplotlib.use('Agg')
        import matplotlib.pyplot as plt

```

(continues on next page)

(continued from previous page)

```

times = np.array([h[0] for h in hist])
true_vals = np.vstack([h[1] for h in hist])
est_vals = np.vstack([h[2] for h in hist])
meas = np.hstack([h[4][0] for h in hist])

fig, ax = plt.subplots(3, 1, figsize=(8, 8), sharex=True)
ax[0].plot(times, true_vals[:,0], label='true omega')
ax[0].plot(times, est_vals[:,0], label='est omega', linestyle='--')
ax[0].plot(times, meas, label='measurement', linestyle=':', alpha=0.6)
ax[0].legend()
ax[0].set_ylabel('omega')

ax[1].plot(times, true_vals[:,1], label='true a')
ax[1].plot(times, est_vals[:,1], label='est a', linestyle='--')
ax[1].legend()
ax[1].set_ylabel('a')

ax[2].plot(times, true_vals[:,2], label='true b')
ax[2].plot(times, est_vals[:,2], label='est b', linestyle='--')
ax[2].legend()
ax[2].set_ylabel('b')
ax[2].set_xlabel('time [s]')

plt.tight_layout()
out_path = 'ekf_plot.png'
fig.savefig(out_path, dpi=150)
print(f'Plot saved to {out_path}')
except Exception as e:
    print('Plotting failed:', e)

```

